Trabajo Práctico II: Principios SOLID en el Sistema de Notificaciones

Materia: Programación Orientada a Objetos / Diseño de Software
Tema: Rediseño aplicando Principios SOLID
Caso: Sistema de Notificaciones del Hospital Central

1. Diagnóstico del Diseño Inicial (Parte A)

Al analizar la solución previa, el problema central era que toda la lógica de notificaciones estaba concentrada en un único bloque de código (típicamente una clase que decidía mediante condicionales qué medio usar). Este enfoque genera varios problemas claros:

Principios SOLID que se estaban violando
Principio de Responsabilidad Única (SRP): La clase principal se encargaba tanto de gestionar los turnos médicos como de formatear y enviar mensajes por distintos canales (Email, SMS, WhatsApp). Tenía más de un motivo para cambiar: si cambiaba la regla del turno, cambiaba la clase; si cambiaba la forma de conectarse al servicio de SMS, también.

Principio de Abierto/Cerrado (OCP): Cada vez que surgía la necesidad de agregar un nuevo canal de comunicación, era obligatorio abrir la clase existente y modificarla agregando otro bloque if-else o switch. El código no estaba cerrado a la modificación.

Principio de Inversión de Dependencias (DIP): El gestor de turnos dependía directamente de implementaciones concretas en lugar de interactuar a través de una abstracción o contrato común.

Por qué el diseño era frágil y costoso de mantener
Riesgo de regresión: Cada cambio en el archivo principal ponía en riesgo funcionalidades que ya estaban probadas y funcionando correctamente.

Alto acoplamiento: Los módulos de alto nivel estaban atados a detalles de bajo nivel.

Mayor costo del cambio: Cualquier pedido de un nuevo canal exigía reescribir código existente, retestear todo el flujo de turnos y aumentaba la probabilidad de introducir errores.

2. Refactor y Solución Propuesta (Partes B, C y D)

Para resolver estos problemas, rediseñamos la arquitectura separando responsabilidades y desacoplando componentes:

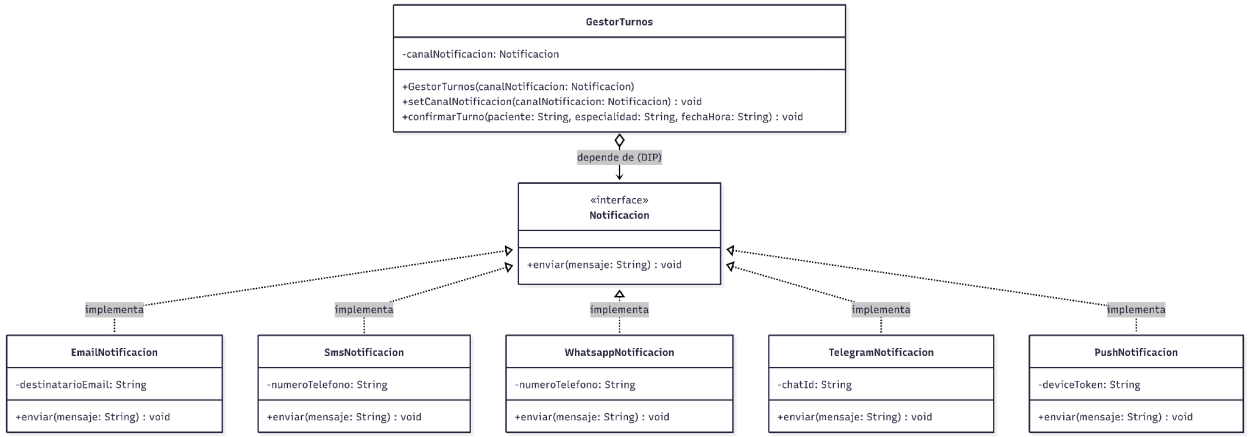
Definición de la abstracción (Notificacion): Creamos una interfaz con un método común enviar(String mensaje). Esto establece el contrato que cualquier canal debe cumplir.

Clases concretas independientes: Cada canal (EmailNotificacion, SmsNotificacion, WhatsappNotificacion, TelegramNotificacion, PushNotificacion) implementa la interfaz y se hace cargo únicamente de su lógica específica.

Aplicación de DIP en GestorTurnos: La clase GestorTurnos ahora recibe una instancia de Notificacion mediante su constructor o mediante un método setter. De esta manera, el gestor no sabe ni le importa si el mensaje sale por Email, SMS o Telegram; solo sabe que el objeto recibido sabe enviar.

Extensibilidad lograda
Cuando el hospital solicitó sumar Telegram y Push Notifications, simplemente creamos dos archivos nuevos que implementan Notificacion. No tuvimos que modificar ni una sola línea de GestorTurnos ni de las notificaciones anteriores, cumpliendo estrictamente con el principio Open/Closed.

3. Diagrama de Clases (UML)



4. Conclusión

El rediseño permitió transformar un sistema rígido en una estructura modular y flexible:
Cumplimos con SRP aislando la lógica de cada canal.
Cumplimos con OCP al poder agregar nuevos medios sin editar código previo.
Cumplimos con DIP y Polimorfismo haciendo que GestorTurnos dependa únicamente de la interfaz Notificacion.
Esto reduce la deuda técnica, simplifica los tests unitarios y baja drásticamente el costo de mantenimiento del sistema a futuro.
Podés pegar este texto directo en tu procesador de texto o Google Docs y ya te queda listo con la estructura formal para la entrega.